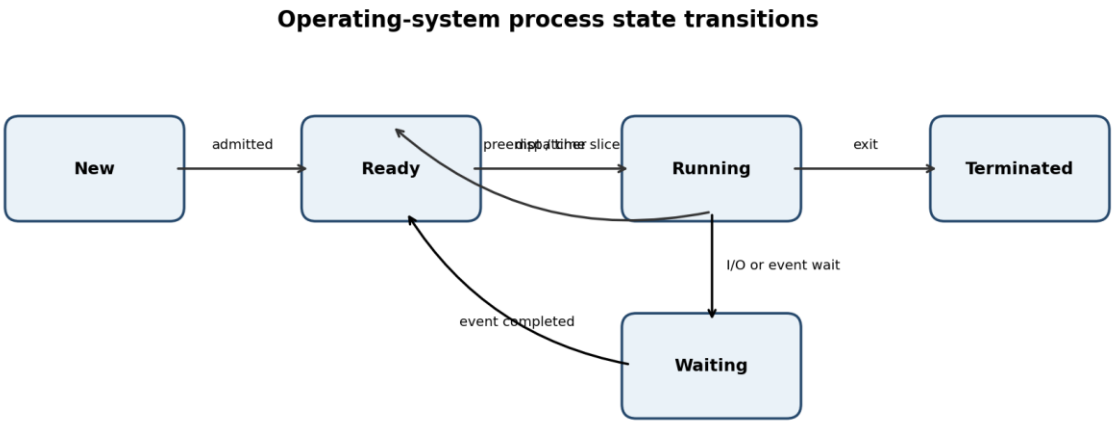
OPERATING SYSTEMS

ASSIGNMENT 2 – CO2

Process Management, Scheduling, Threads, IPC, Synchronisation and Deadlocks



Submitted in partial fulfilment of the Operating Systems course assignment

Student details	Information
Name	S. Maruthu Pandi
Register Number	192511162
Course	CSA0421 – Operating Systems
Assignment	Assignment 2 – CO2
GitHub Repository	https://github.com/Maruthupandi-maker/CSA0421-Operating-System-tree/main/CSA0421%20192511162%20Assignment%202

Q4 – INTER-PROCESS COMMUNICATION

4.1 IPC problem and models

The weather station has a sensor-reader process that collects readings and an analyser process that calculates information such as average and maximum temperature. IPC allows the processes to exchange those readings even though their address spaces are separate.

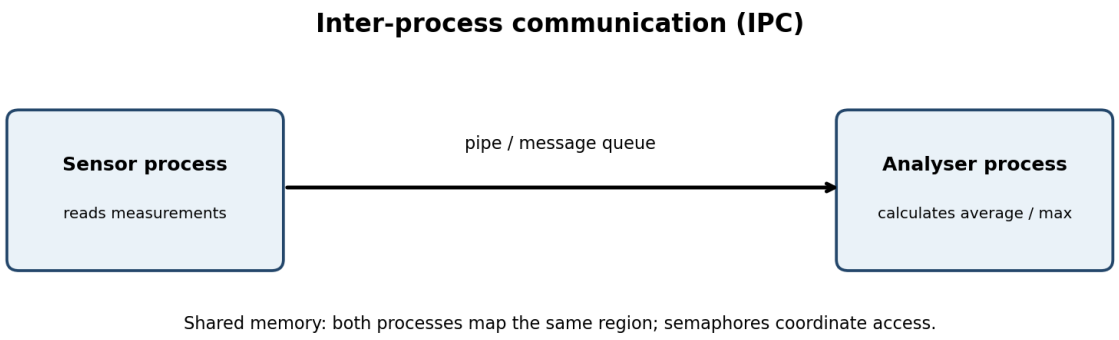


Figure 4. Sensor-reader and analyser processes communicate through IPC.

IPC method	How it works	Copying and synchronisation
Pipe	A kernel-managed byte stream connects a writer and a reader, commonly between related processes.	The kernel copies data between user buffers and the pipe buffer. Blocking read/write provides flow control; closing all write descriptors signals EOF.
Message queue	Processes send and receive discrete messages through a kernel-managed queue.	Kernel transfers message data; messages retain boundaries and can carry priorities. Queue operations provide synchronisation.
Shared memory	Two or more processes map the same physical memory region into their address spaces.	Processes access the common region without copying every data item through a pipe. They must use semaphores, mutexes or atomics to avoid races.

4.2 Pipe implementation – completed TODO

The parent writes six integer readings into the pipe. The child closes its unused write end, reads one integer at a time, and calculates count, sum and maximum. The parent closes its write descriptor after sending the final reading. When every write descriptor is closed and buffered bytes are consumed, read() returns 0 to the child.

```c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    int fd[2];
    if (pipe(fd) == -1) { perror("pipe"); return 1; }
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {
        close(fd[1]);
        int r, count = 0, sum = 0, max = 0;
        ssize_t n;
        while ((n = read(fd[0], &r, sizeof r)) > 0) {
            if (n != (ssize_t)sizeof r) {
                fprintf(stderr, "partial integer received\n");
                close(fd[0]); _exit(2);
            }
            if (count == 0 || r > max) max = r;
            sum += r; ++count;
        }
        if (n < 0) perror("read");
        printf("analyser: %d readings, average %.1f, max %d\n",
               count, count ? (double)sum / count : 0.0, max);
        close(fd[0]); fflush(stdout); _exit(0);
    }
    close(fd[0]);
    int readings[] = {31, 33, 36, 34, 41, 38};
    for (int i = 0; i < 6; ++i)
        if (write(fd[1], &readings[i], sizeof readings[i]) != (ssize_t)sizeof readings[i])
            perror("write");
    close(fd[1]);
```

```
    waitpid(pid, NULL, 0);
    printf("reader: done\n");
    return 0;
}
```

Compile and execute:

```
gcc -std=c11 -Wall -Wextra q4_pipe.c -o q4_pipe
./q4_pipe
```

Expected output:

```
analyser: 6 readings, average 35.5, max 41
reader: done
```

4.3 Why closing the write end is essential

If close(fd[1]) is removed in the parent, the parent still owns an open write descriptor. After the six integers are read, the child calls read() again. The pipe is empty, but the kernel cannot report end-of-file because a writer is still open. The child blocks waiting for more data, and the parent waits for the child with waitpid(). This creates a hang. Closing unused pipe ends is therefore part of the IPC protocol, not merely resource cleanup.

4.4 Shared-memory implementation using process-shared semaphores

The following second implementation maps one shared ring buffer into both processes. The empty semaphore counts available buffer slots and the full semaphore counts stored readings. The parent waits for an empty slot before writing; the child waits for a full slot before reading. These semaphores use pshared=1 so they can synchronise processes sharing the mapped memory.

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <semaphore.h>

#define CAP 8
typedef struct {
    sem_t empty, full;
    int data[CAP], in, out;
} Shared;

int main(void) {
    Shared *s = mmap(NULL, sizeof(*s), PROT_READ | PROT_WRITE,
                     MAP_SHARED | MAP_ANONYMOUS, -1, 0);
    if (s == MAP_FAILED) { perror("mmap"); return 1; }
    s->in = s->out = 0;
    if (sem_init(&s->empty, 1, CAP) || sem_init(&s->full, 1, 0)) {
        perror("sem_init"); munmap(s, sizeof(*s)); return 1;
    }
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {
        int count = 0, sum = 0, max = 0;
        int total = 6;
        for (int k = 0; k < total; ++k) {
            sem_wait(&s->full);
            int value = s->data[s->out];
            s->out = (s->out + 1) % CAP;
            sem_post(&s->empty);
            if (count == 0 || value > max) max = value;
```

```
            sum += value; ++count;
        }
        printf("shared analyser: %d readings, average %.1f, max %d\n",
               count, (double)sum/count, max);
        fflush(stdout); _exit(0);
    }
    int readings[] = {31, 33, 36, 34, 41, 38};
    for (int i = 0; i < 6; ++i) {
        sem_wait(&s->empty);
        s->data[s->in] = readings[i];
        s->in = (s->in + 1) % CAP;
        sem_post(&s->full);
    }
    waitpid(pid, NULL, 0);
    sem_destroy(&s->empty); sem_destroy(&s->full);
    munmap(s, sizeof(*s));
    return 0;
}
```

Compile and execute the shared-memory version:

```
gcc -std=c11 -Wall -Wextra -pthread q4_shm.c -o q4_shm
./q4_shm
```

Both examples should calculate an average of 35.5 and maximum of 41. The shared-memory example is a single-producer/single-consumer demonstration. For production use, check every sem_wait/sem_post return value and add robust error handling and shutdown coordination.

4.5 Pipe versus shared memory

Comparison	Pipe	Shared memory + semaphore
Data path	Writer copies data into a kernel pipe buffer; reader copies it to its user buffer.	Both processes read/write the mapped shared region directly.

Programming difficulty	Simple sequential byte stream; kernel manages blocking.	More complex; buffer layout, ownership and synchronisation are application responsibilities.
Synchronisation	Blocking pipe operations and descriptor closure coordinate flow.	Explicit semaphores or mutexes are required.
Good use	Small or moderate streams, parent-child communication and command pipelines.	High-volume data exchanged repeatedly between processes on the same machine.

4.6 Growth investigation: 1 MB, 10 MB and 100 MB

To compare transfer time, keep the payload, machine, compiler options and number of runs the same. Run each test several times and report the median or average. For a pipe, measure the total time until the consumer receives the complete payload. For shared memory, measure writing/reading the same number of bytes and include the semaphore coordination cost.